Day 4: Time Series Analysis

Naeemullah Khan
naeemullah.khan@kaust.edu.sa

جامعة الملك عبدالله
للعلوم والتقنية
King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

April 23, 2026

Learning Outcomes

By the end of this session, you will be able to:

1. Identify the key characteristics of time series data (trend, seasonality, stationarity) and how they differ from cross-sectional data.
2. Use the ACF and PACF to identify appropriate AR, MA, and ARIMA models.
3. Apply the full ARIMA workflow: identify, estimate, diagnose, and forecast.
4. Handle seasonal data using SARIMA and decomposition methods.
5. Analyze relationships between two time series using cross-correlation and lagged regression.

Table of Contents

From Regression to Time Series

Days 1–3 Recap: All our models assumed **independent** observations.

Cross-Sectional Data

- Each row is independent
- Order doesn't matter
- Shuffle the rows → same result

Example: House prices dataset

Time Series Data

- Observations are **ordered** in time
- **Nearby** values are correlated
- Shuffle → destroys the pattern

Example: Daily stock prices

Key shift: Today, *the order of observations carries information.*

What is a Time Series?

Definition

A **univariate time series** is a sequence of measurements of the same variable collected over time, usually at regular intervals.

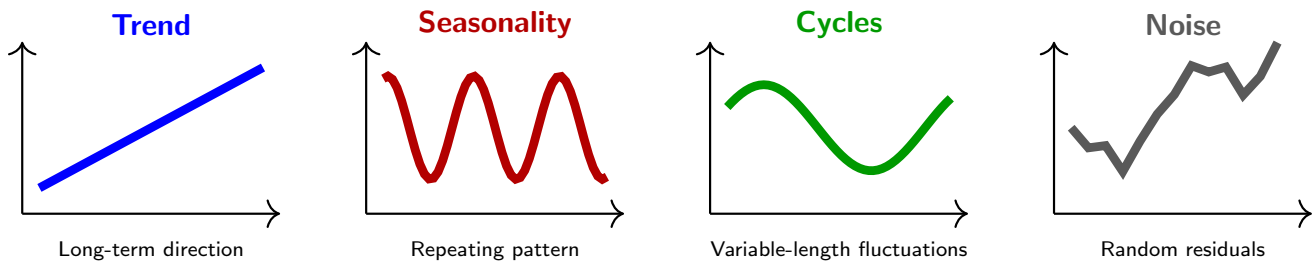
Examples are everywhere:

Finance	**Science**	**Business**
Stock prices, exchange rates, revenue	Temperature, rainfall, CO_2 levels	Monthly sales, web traffic, demand

Notation: $x_1, x_2, \ldots, x_n$ where x_t is the value at time t.

Components of a Time Series

Every time series can be thought of as a combination of components:



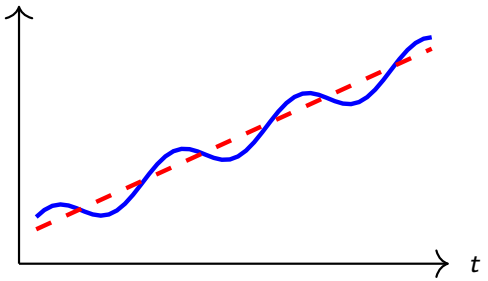
The goal of time series analysis: Decompose the data into these components, model the structure, and use it to **forecast** future values.

Trend

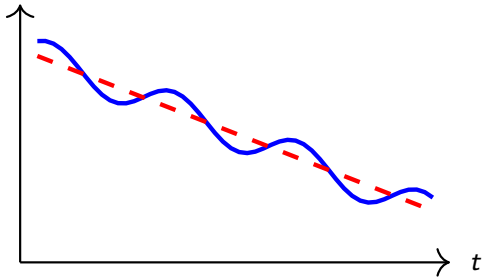
Definition

A **long-term upward or downward movement** in the data, indicating a general increase or decrease over time.

Upward Trend



Downward Trend



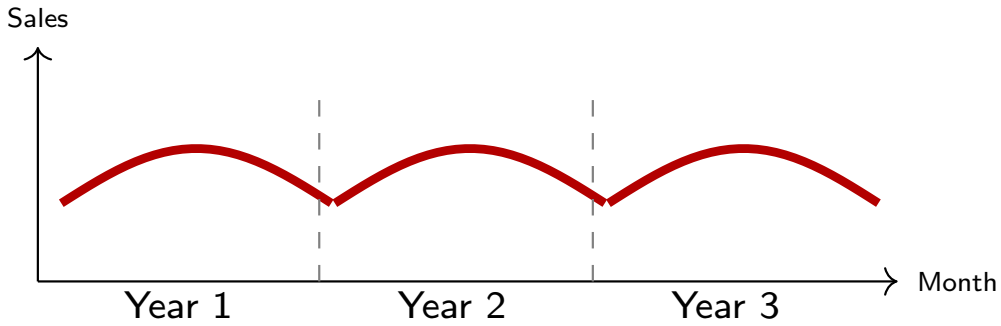
Examples:

- Global average temperature rising over decades
- Company revenue growing year over year
- Population of a city increasing over time

Seasonality

Definition

A **repeating pattern** that occurs at **regular, known intervals** — daily, weekly, monthly, or yearly.



Key property: The period S is **fixed and known** (e.g., $S = 30$ for monthly, $S = 7$ for daily).

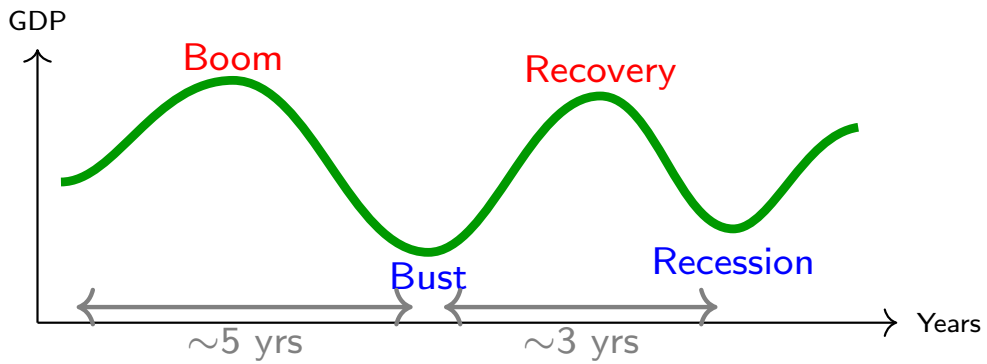
Examples:

- Ice cream sales peak every summer
- Website traffic drops every weekend

Cycles

Definition

A pattern that **rises and falls** over time, but **not at a fixed period**. The length varies and is often unpredictable.



Seasonality

Fixed period, calendar-driven
Every December, every summer
S is known (12, 7, 24. . .)

Cycle

Variable length, economy/nature-driven
Recessions, sunspot cycles
Period changes each time

Autocorrelation, Outliers, and Noise

Autocorrelation

Correlation between an observation and a **previous** one.

Today's temp. is correlated with yesterday's.

What makes TS special!

Outliers

Extreme observations **significantly different** from the rest.

A sensor glitch, a flash crash.

Can distort models if not handled.

Noise

Random, unpredictable variations.

Measurement error, random shocks.

Goal: model everything except noise.

Summary: Time series = Trend + Seasonality + Cycles + Autocorrelation + Outliers + Noise.

Our job: model the structure, detect outliers, accept noise as irreducible.

What is a Lag? (1/3)

Definition

A **lag** is a time-shifted copy of a series. Lag h means "the value from h time steps ago."

$$x_{t-h} \quad \text{is the value of } x \text{ at time } t-h$$

Example: Monthly temperature readings.

Month t	1	2	3	4	5	6
x_t (original)	20	22	19	25	23	21
x_{t-1} (lag 1)	—	20	22	19	25	23
x_{t-2} (lag 2)	—	—	20	22	19	25

What is a Lag? — Adding Lags to a Dataset (2/3)

In practice, we create lag columns in our dataset and use them as features.

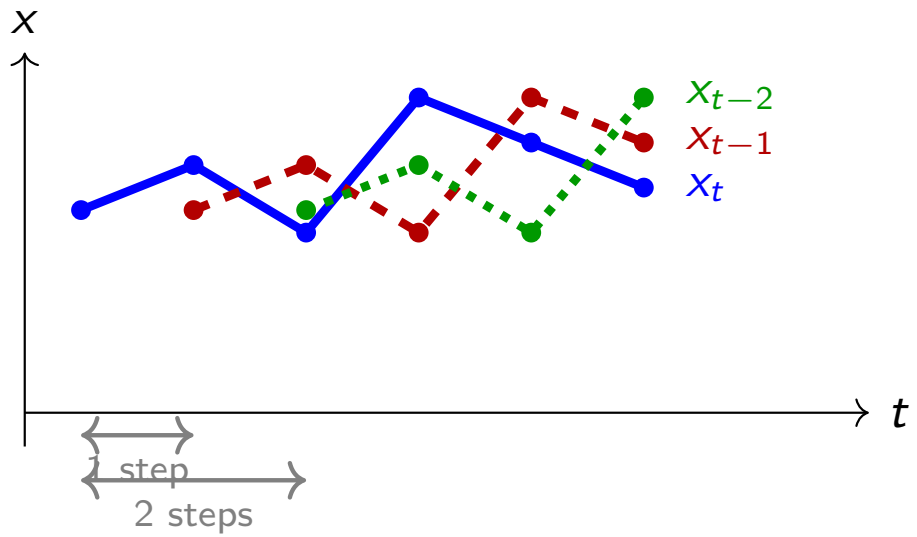
Month t	x_t **(temp)**	x_{t-1} **(lag 1)**	x_{t-2} **(lag 2)**	**Use?**
1	20	`NaN`	`NaN`	✗ Drop
2	22	20	`NaN`	✗ Drop
3	19	22	20	✓ Keep
4	25	19	22	✓ Keep
5	23	25	19	✓ Keep
6	21	23	25	✓ Keep

What happened row by row:

- **Row 1:** No previous values exist at all — both lag columns are `NaN`. Drop it.
- **Row 2:** Lag 1 is filled (month 1), but lag 2 still has no value. Drop it.
- **Rows 3–6:** Both lag columns are complete. These rows are ready to use.

What is a Lag? (3/3)

Visualising the shift: each lagged series is the original pushed rightward.



Why lags matter: Autocorrelation measures how correlated x_t is with x_{t-h} — how much does knowing the past help predict the present?

The **ACF** and **PACF** are simply plots of these correlations across all lags $h = 1, 2, 3, \ldots$

What is Differencing? (1/2)

Definition

Differencing means replacing each value with the **change** from the previous value:

$$x'_t = x_t - x_{t-1}$$

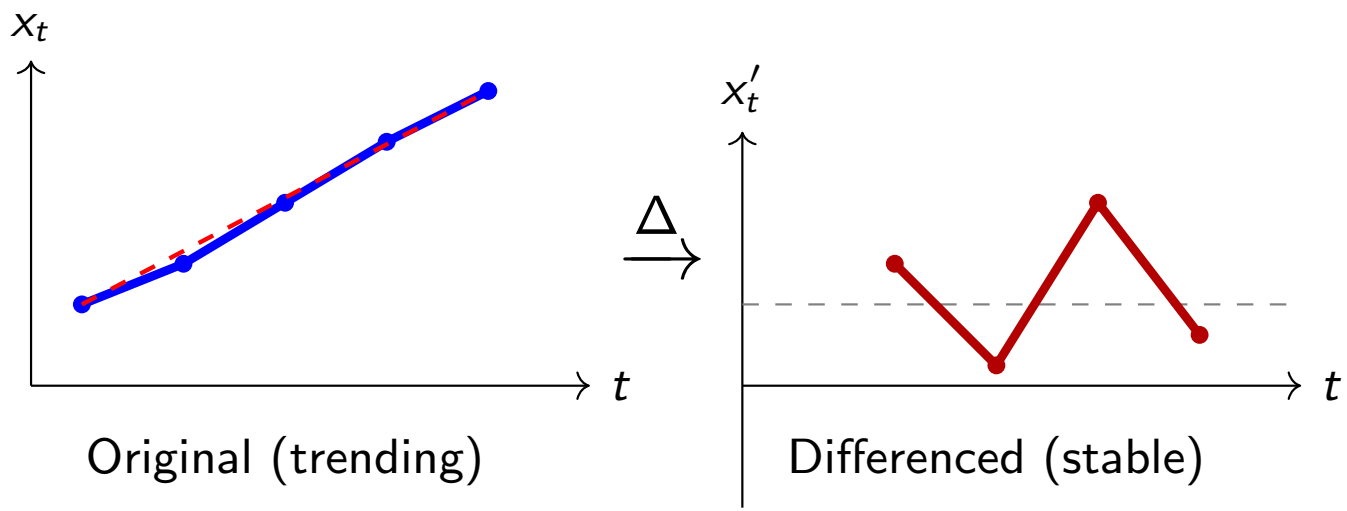
Instead of asking "what is the value today?" we ask "how much did it change from yesterday?"

Example: Monthly temperature readings.

Month t	x_t **(original)**	x'_t **(differenced)**	**Meaning**
1	20	—	no previous value
2	22	+2	rose by 2
3	19	−3	fell by 3
4	25	+6	rose by 6
5	23	−2	fell by 2

What is Differencing? (2/2)

Visualising the effect: differencing removes the trend and leaves only the changes.



Why it matters: A trending series has a mean that keeps changing — hard to model. After differencing, we look at **changes** that fluctuate around a stable level. This is what models like ARIMA require.

Why Do We Need Stationarity?

أكاديمية كاوست
KAUST ACADEMY

The Problem

Most time series models (AR, MA, ARIMA) assume the **statistical properties don't change over time**. If the mean drifts or variance expands, parameter estimates become **unreliable**.

Think of it this way:

- In regression, you assume the relationship between x and y is **stable**
- In time series, the relationship between x_t and x_{t-1} must be stable too
- If the mean keeps rising, yesterday's correlation structure won't apply tomorrow

Analogy: It's like aiming at a target that's slowly moving you can't tell if your aim is off or the target is drifting. Make the target stationary first, then evaluate your accuracy

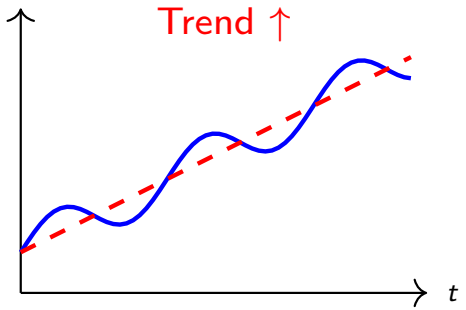
Stationarity: The Key Assumption

Weakly Stationary

A time series is **stationary** if its statistical properties do not change over time:

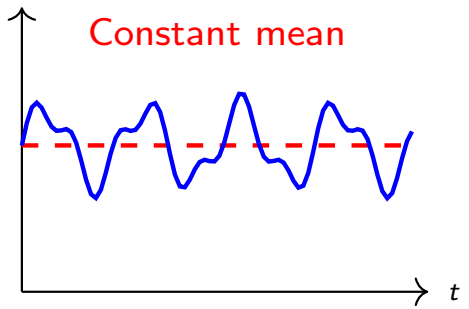
- Constant **mean** (no trend)
- Constant **variance** (no expanding/shrinking spread)
- **Autocorrelation** depends only on lag, not on time

Non-Stationary



Mean changes over time

Stationary



Fluctuates around a fixed level

How to detect: A slowly decaying ACF is the classic sign of non-stationarity.

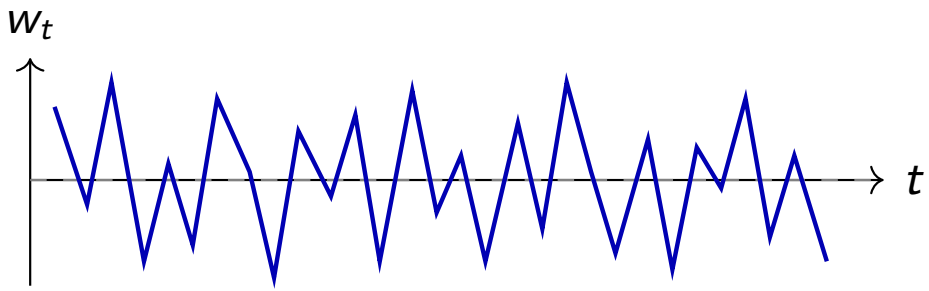
White Noise: The Baseline

Definition

A **white noise** process $\{w_t\}$ is the simplest possible time series:

- Zero mean: $E[w_t] = 0$
- Constant variance: $\text{Var}(w_t) = \sigma^2$
- No autocorrelation: $\text{Corr}(w_t, w_s) = 0$ for all $t \neq s$

What it looks like:

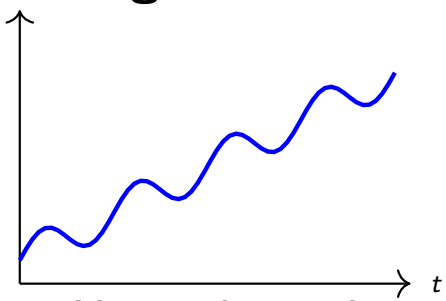


No trend, no pattern, no memory — every value is a fresh random draw. This is exactly what **model residuals should look like**.

Making a Series Stationary: Differencing

Problem: Series has a trend $\rightarrow$ non-stationary.
Solution: First differencing $x'_t = x_t - x_{t-1}$

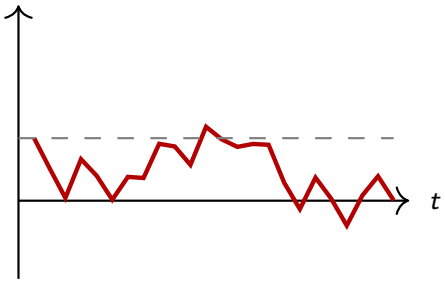
Original Series



Upward trend

$\xrightarrow{\Delta}$

After First Differencing



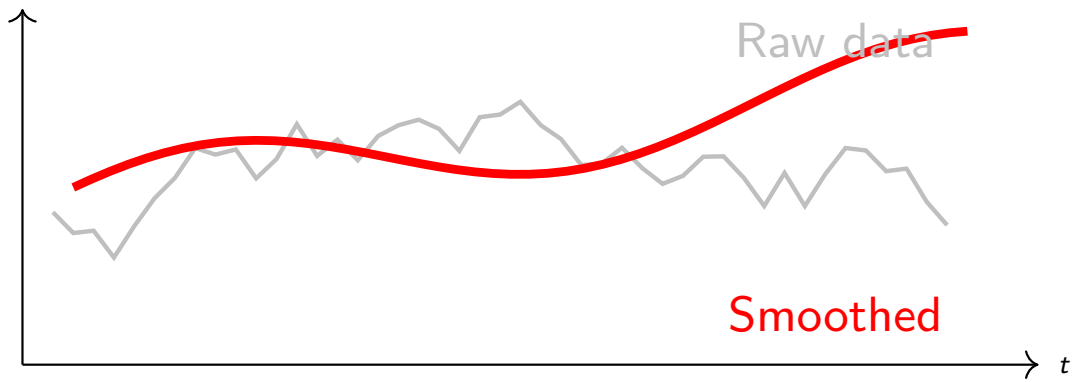
Trend removed!

Why it works: If the series grows by roughly the same amount each step, then the *differences* will fluctuate around a constant — i.e., they're stationary.

Rules: Slowly decaying ACF $\rightarrow$ difference. Rarely need $d > 2$. Always re-check ACF after.

Smoothing: Seeing Through the Noise

Moving Average Smoothing: Replace each value with the average of a window of nearby values.



Window Size Trade-off

Small window:
Follows the data closely but still noisy

Large window:
Very smooth but may lose important detail

Note: This is *not* the MA in ARIMA. This is a data preprocessing tool.

Time Series Decomposition (1/2)

Every time series can be broken into three parts:

$$\text{Original} = \text{Trend} + \text{Seasonal pattern} + \text{Residual noise}$$

The question is: how does the seasonal pattern behave as the series grows?

Additive

The seasonal swings stay the **same size** regardless of the level.

Example: A shop always sells 100 more units every December, whether annual sales are 1,000 or 10,000.

$x_t = T_t + S_t + R_t$

Multiplicative

The seasonal swings **grow** as the series grows.

Example: A shop always sells 20% more in December. As the business grows, that 20% becomes a bigger and bigger number.

$x_t = T_t \times S_t \times R_t$

ACF — The "Total Memory" Test (1/3)

What is the ACF?

The **Autocorrelation Function** at lag h measures the **total correlation** between x_t and x_{t-h} — how much does the value today relate to the value h steps ago?

Intuition — the pond ripple:

- Drop a pebble in a still pond. The first ripple (lag 1) is strong.
- The second ripple (lag 2) is weaker, but still moving — because ripple 1 pushed it.
- The ACF measures **total movement** at every ripple, including the indirect push.
- It does **not** separate "direct" from "passed-along" correlation.

ACF — The Formula (2/3)

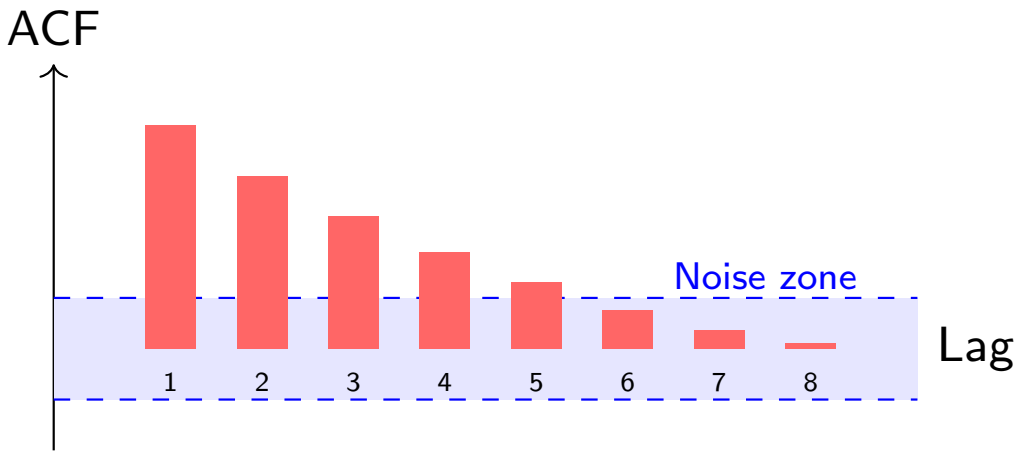
أكاديمية كاوست
KAUST ACADEMY

Formula

$$r_h = \frac{\sum_{t=h+1}^{n} (x_t - \bar{x})(x_{t-h} - \bar{x})}{\sum_{t=1}^{n} (x_t - \bar{x})^2}$$

This is exactly **Pearson correlation**, but between the series x_t and its own copy shifted h steps back.

ACF — How to Read the Plot (3/3)



Three patterns to recognise:

- **Slow slide** (bars decay gradually over many lags): data has a **trend**, not stationary. *Action: difference first, then restart.*
- **Sharp drop** (bars vanish after lag q): likely an **MA(q)** model. Count the bars outside the blue zone — that number is q.
- **All bars inside the blue band**: no significant autocorrelation — this is **white noise**. Exactly what you want to see in your residuals.

PACF — The "Direct Connection" Test (1/2)

أكاديمية كاوست
KAUST ACADEMY

What is the PACF?

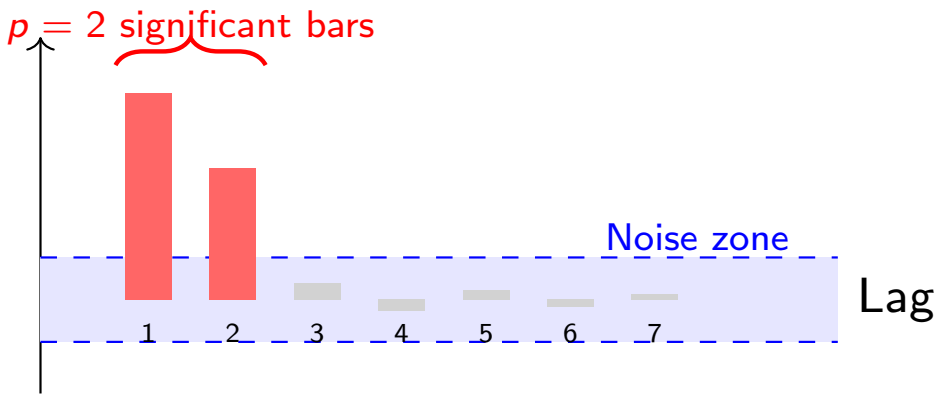
The **Partial Autocorrelation Function** at lag k measures the **direct** correlation between x_t and x_{t-k}, **after removing** the influence of all observations in between.

Intuition — the telephone game:

- Person 1 whispers to Person 2, who whispers to Person 3.
- The ACF asks: "Is Person 1 correlated with Person 3?" — Yes, but only *through* Person 2.
- The PACF **mutes Person 2** and asks: "Does Person 1 *directly* affect Person 3?"
- If no — the PACF at lag 2 is zero. The connection was never direct.

PACF — The "Direct Connection" Test (2/2)

How to read a PACF plot (AR(2) example):



"Sharp drop" after lag p:
You have an **AR(p)** model.
Count the bars outside the noise zone.

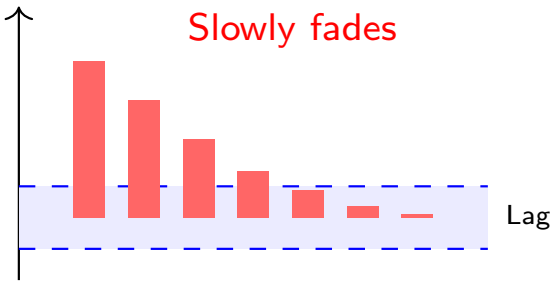
"Slow oscillating decay":
Consistent with an **MA** model — an AR model needs infinitely many terms to mimic MA behavior.

Key rule: ACF identifies **MA** models (sharp cutoff in ACF). PACF identifies **AR** models (sharp cutoff in PACF). They are mirror images of each other.

Example: The AR Pattern — What You See

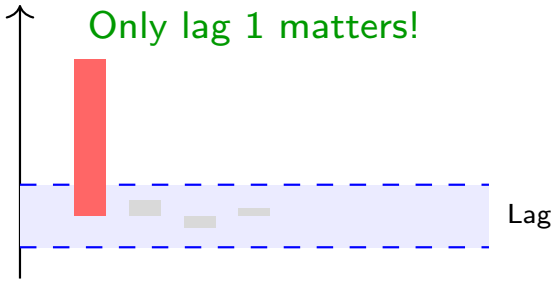
Scenario: Daily temperature in London.
If today is 20°C, tomorrow is likely 18–22°C. There is "momentum."

ACF: Slow Decay



Slopes down gradually:
$0.8 \to 0.6 \to 0.4 \to \ldots$

PACF: Sharp Drop



Lag 1 is large, then **noise only**.

Example: The AR Pattern — Interpretation

Why does the ACF decay slowly but the PACF cuts off?

- **ACF at lag 2:** Today's temp $\leftrightarrow$ temp 2 days ago. This looks correlated because *yesterday* links them — not because day t and day $t-2$ are directly related.
- **PACF at lag 2:** Once we control for yesterday's temperature, the day-before-yesterday adds **no new information**. PACF goes to zero.
- The PACF strips away the "telephone game" effect and shows only **direct** relationships.

Diagnosis: AR(1).
PACF cuts off at lag 1. ACF decays exponentially. Once we know today's temperature, yesterday's is the only past value that directly matters.

Moving Average Models: The Core Idea (1/2)

أكاديمية كاوست
KAUST ACADEMY

The idea

An MA model says: today's value is affected by recent **surprises** — things that were **unexpected** compared to what the model predicted.

What is a "surprise"?

Every day, the model makes a prediction. The **surprise** is simply:

$$\text{surprise} = \text{actual value} - \text{predicted value}$$

AR remembers past *values*:
"Yesterday's price affects today's price."

MA remembers past *surprises*:
"Yesterday's unexpected jump affects today's price."

Moving Average Models: The Core Idea (2/2)

Example: A stock was predicted to gain \$5 today, but gained \$12 instead.

The surprise $= +7$. An MA model says this surprise **echoes** into the next day.

Day	Surprise still felt?	Amount
Today	Full surprise	$+7.0$
Tomorrow	70% of it lingers	$+4.9$
Day after	Gone completely	**0**

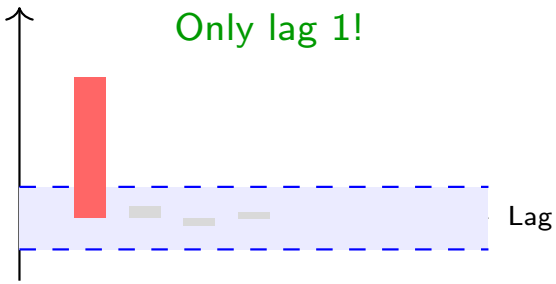
Key point: The surprise does not linger forever.

After a fixed number of days q, it is **completely forgotten**.
This is what makes MA models different from AR — the memory is **short and exact**, not a gradual decay.

Example: The MA Pattern — What You See

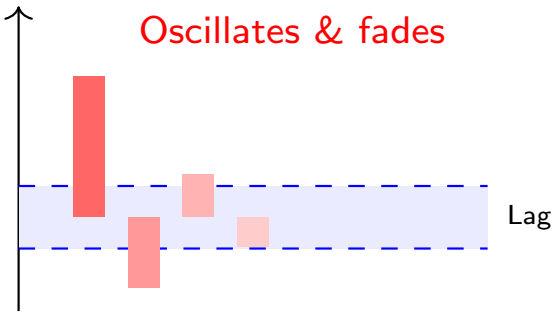
Scenario: Daily stock returns (the *change* in price).

ACF: Sharp Drop



One spike at lag 1, then **nothing**.

PACF: Slow Oscillating Decay



Alternates up/down, slowly shrinks.

Example: The MA Pattern — Interpretation

Why does the ACF cut off but the PACF oscillates?

- **ACF at lag 1:** Today's return and yesterday's share the common shock w_{t-1}. Clear correlation.
- **ACF at lag 2:** Today's return (w_t, w_{t-1}) and two days ago (w_{t-2}, w_{t-3}) share **no shocks at all**. Correlation is exactly zero.
- **PACF oscillates:** When you try to fit an AR model to MA data, you need infinitely many AR terms to approximate it — the PACF slowly dies across many lags instead of cutting sharply.

Diagnosis: MA(1).
A news shock affects returns for exactly one day, then the market "forgets" completely. ACF cutoff at lag 1 is the signature.

The Detective's Cheat Sheet

Find the plot that "dies" (cuts off sharply):

You see...	Model	How to find order
ACF tails off slowly PACF cuts off sharply	**AR(p)**	p = significant bars in PACF
ACF cuts off sharply PACF tails off slowly	**MA(q)**	q = significant bars in ACF
Both tail off slowly	**ARMA**	Start with (1,1), experiment

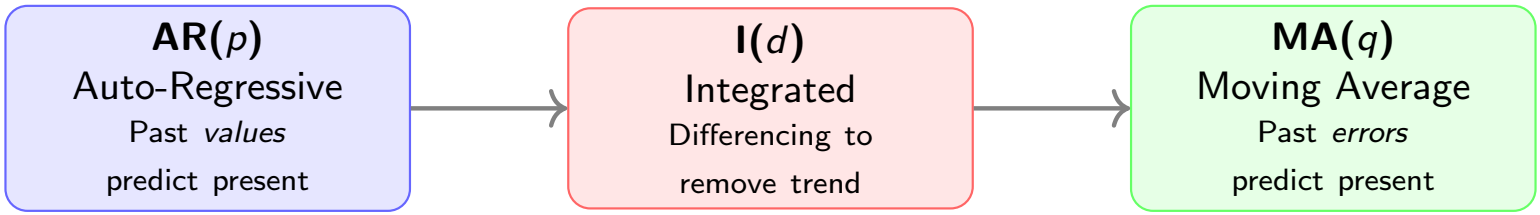
Pro Tip: If the ACF stays large for many lags $\rightarrow$ your data is **non-stationary**. Do not look at PACF yet. Go back, **difference it**, and start over.

Summary:

ACF $\rightarrow$ MA models + stationarity check PACF $\rightarrow$ AR models

The ARIMA Family: Overview

ARIMA stands for **A**uto**R**egressive **I**ntegrated **M**oving **A**verage.
It's a **family** of models built from three building blocks:



ARIMA(p, d, q)

The idea: Start simple, add complexity as needed.
AR only? $\rightarrow$ ARIMA(p,0,0). MA only? $\rightarrow$ ARIMA(0,0,q). Need differencing? $\rightarrow$ increase d.
Seasonal? $\rightarrow$ extend to SARIMA. We'll build up step by step.

AR Models: Predicting from Past Values

أكاديمية كاوست
KAUST ACADEMY

AR(p) — Autoregressive Model

$$x_t = c + \phi_1 x_{t-1} + \phi_2 x_{t-2} + \cdots + \phi_p x_{t-p} + w_t$$

The current value is a **linear combination of its own past values** plus noise.

Simplest case — AR(1): $x_t = c + \phi_1 x_{t-1} + w_t$

- This is **linear regression** where the predictor is the previous value
- ϕ_1 controls how strongly yesterday influences today

ACF: Exponentially decays (tails off). **PACF:** Cuts off after lag p.

Examples:

- Today's temperature $\approx$ yesterday's $\pm$ noise $\rightarrow$ AR(1)
- This quarter's GDP depends on the last 2 quarters $\rightarrow$ AR(2)

MA Models: Predicting from Past Shocks (1/4)

MA(q) — Moving Average Model

$$x_t = \mu + w_t + \theta_1 w_{t-1} + \theta_2 w_{t-2} + \cdots + \theta_q w_{t-q}$$

μ	= the mean of the series
w_t	= current random shock (white noise)
$\theta_i w_{t-i}$	= fraction of a past shock still echoing today

AR vs. MA in one line:

AR: x_t depends on past *values* $x_{t-1}, x_{t-2}, \ldots$

MA: x_t depends on past *shocks* $w_{t-1}, w_{t-2}, \ldots$

MA Models: Predicting from Past Shocks (2/4)

Intuition — a stone thrown into a still pond:

- The stone = a random **shock** w_t
- The ripples = echoes of that shock at lags 1, 2, ..., q
- After lag q, the pond is **perfectly calm again**

AR model

Memory of past *values*
x_t depends on $x_{t-1}, x_{t-2}, \ldots$

"Where I am depends on where I was."

MA model

Memory of past *shocks*
x_t depends on $w_{t-1}, w_{t-2}, \ldots$

"Where I am depends on recent surprises."

MA Models: Predicting from Past Shocks (3/4)

Scenario: A trader receives daily news shocks that affect stock returns.

MA(1) model: x_t

A news shock on Day 1 affects Day 1 fully, then Day 2 at 70%, then is **completely forgotten** by Day 3.

Suppose a shock $w_1 = +2$ hits on Monday:

Day	Contribution from w_1	Still felt?
Monday	$w_1 = +2$	Full impact
Tuesday	$0.7 \times w_1 = +1.4$	70% echo
Wednesday	0	**Gone completely**

MA Models: Predicting from Past Shocks (4/4)

Generalization — MA(q):

$$x_t = \mu + w_t + \theta_1 w_{t-1} + \cdots + \theta_q w_{t-q}$$

The parameter q controls how many past shocks still echo into today.

Why this matters for the ACF:

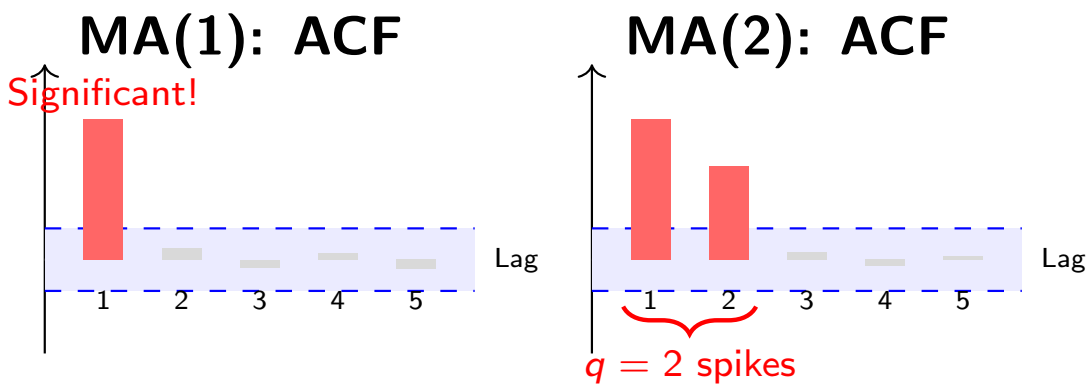
x_t and x_{t-1} share shock w_{t-1} $\rightarrow$ correlated $\Rightarrow$ spike at lag 1

x_t and x_{t-2} share **no shocks at all** $\rightarrow$ correlation is **exactly zero**

This is why the ACF drops to zero after lag q — the diagnostic signature of MA.

ACF Pattern for MA Models

Key property: For an MA(q) model, the ACF is **exactly zero** after lag q.



The pattern: MA(q) $\rightarrow$ ACF has exactly q significant spikes, then **cuts off**. Opposite of AR, where ACF *decays gradually*.

ARIMA: Putting It All Together

ARIMA(p, d, q)

Auto**R**egressive **I**ntegrated **M**oving **A**verage

p = number of AR terms (past values)
d = order of differencing (to achieve stationarity)
q = number of MA terms (past errors)

Examples:

- ARIMA(1,0,0) = AR(1) — no differencing, no MA
- ARIMA(0,1,1) = difference once, then fit MA(1)
- ARIMA(1,1,1) = difference once, then AR(1) + MA(1)
- ARIMA(0,0,0) = white noise (no structure at all)

The "I" in ARIMA handles the non-stationarity we discussed earlier.

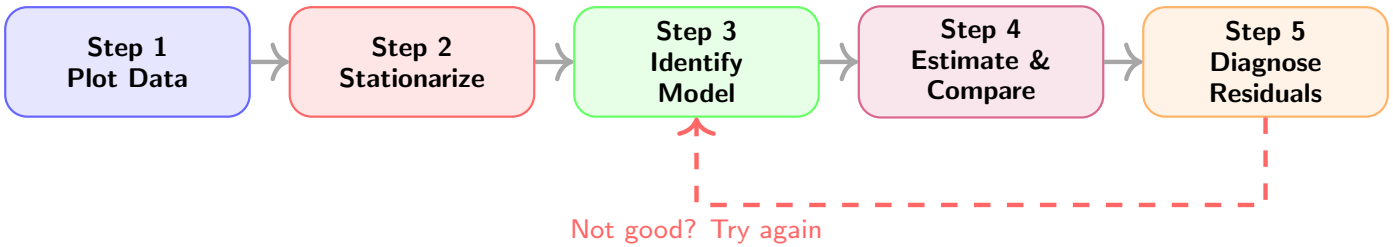
Decomposition vs. ARIMA

	ARIMA	**Decomposition**
Purpose	Flexible modeling	Understanding data
Complexity	Moderate	Simple
Prediction Intervals	Yes	No
Best For	Accurate forecasting	Exploratory analysis

In practice:

Start with **decomposition** to understand the structure of your data → then build **ARIMA** for a rigorous model and forecasts.

The ARIMA Workflow: 5 Steps



Step 1: Look for trends, seasonality, outliers, non-constant variance.

Step 2: Difference if needed. Check ACF of differenced series.

Step 3: Use ACF/PACF cheat sheet to pick candidate models.

Step 4: Fit candidates. Check significance of coefficients. Compare AIC/BIC (lower = better).

Step 5: ACF of residuals should show *no* significant correlations. If it does $\rightarrow$ back to Step 3.

Workflow Example: Steps 1–2

Example: Annual water level of Lake Erie ($n = 40$ years).

Step 1: Plot the data

- No obvious trend
- No clear seasonality (annual data)
- Possibly some cyclical behavior

Step 2: Check stationarity

- ACF decays fairly quickly
- No differencing needed ($d = 0$)

[Time series plot shown in lab]

Key observation:
ACF has a significant spike at lag 1, then drops $\rightarrow$ suggests AR(1)

PACF confirms: spike at lag 1, then cuts off

Workflow Example: Steps 3–4

Step 3: Identify candidate models
From ACF/PACF: try AR(1), possibly MA(1), and ARMA(1,1)

Step 4: Estimate and compare

Model	AIC	Coeff. Significant?	Verdict
AR(1)	105.3	ϕ_1: Yes	**Good**
MA(1)	112.7	θ_1: Yes	Worse fit
ARMA(1,1)	106.1	ϕ_1: Yes, θ_1: No	Over-parameterized

Winner: AR(1)
- Lowest AIC
- All coefficients significant
- ARMA(1,1) adds a parameter that isn't needed

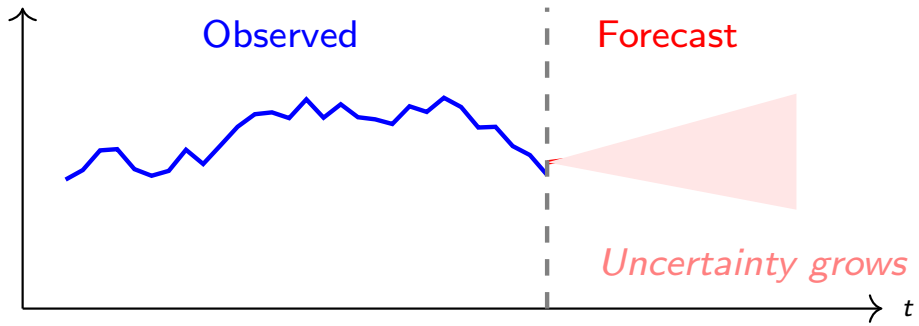
Workflow Example: Step 5 & Forecasting

Step 5: Check residuals

- ACF of residuals: no significant spikes $\rightarrow$ **Good!**
- Ljung-Box test: $p > 0.05 \rightarrow$ no remaining autocorrelation
- Residuals look like white noise $\rightarrow$ model is adequate

Forecasting:

Once validated, generate forecasts. **Near future:** narrow confidence band. **Far future:** uncertainty grows (wider band). The model honestly tells you how uncertain it is.



Model Selection: AIC and BIC

You've seen these before in regression! Same idea here.

AIC (Akaike Information Criterion): Balances goodness of fit vs. complexity. **Lower = better.**
BIC (Bayesian Information Criterion): Similar, but penalizes complexity more heavily.

In practice:

1. Fit 2–3 candidate models
2. Compare their AIC (and/or BIC) values
3. Choose the model with the lowest AIC
4. Verify: are all coefficients significant? Are residuals clean?

Pitfall: Don't just blindly minimize AIC. Always check residuals and make sure the model makes sense for your data.

Common Pitfalls

Overfitting

Adding too many AR/MA terms.
Symptom: Good fit, poor forecasts.
Fix: Use AIC/BIC, check significance.

Skipping Residual Check

Not diagnosing residuals.
Symptom: Residual ACF has spikes.
Fix: Try a different model.

Ignoring Non-Stationarity

Fitting ARMA to a trending series.
Symptom: Slowly decaying ACF.
Fix: Difference first.

Non-Constant Variance

Variance grows with the level.
Symptom: Fan-shaped residuals.
Fix: Log transform or ARCH models.

Residuals: What They Are

Definition

A **residual** is the difference between what your model predicted and what actually happened:

$$e_t = x_t - \hat{x}_t$$

After fitting a model, residuals are what is **left over** — the part the model could not explain.

Why do we care?

- If the model captured all structure, residuals should look like **pure noise** — no patterns left.
- If residuals still show patterns, the model **missed something** and needs improvement.
- Residual analysis is the primary way we verify a model is adequate.

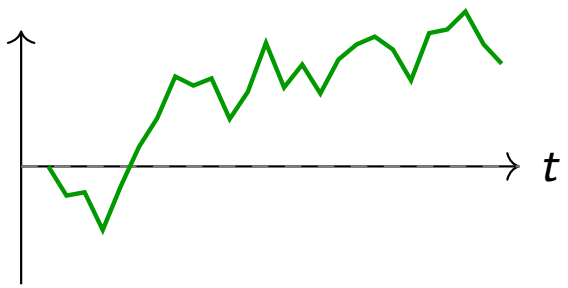
What Good Residuals Look Like

Good residuals

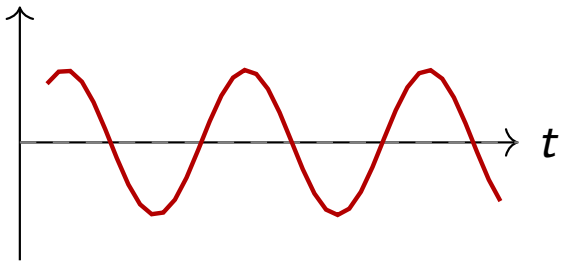
- Mean of zero
- Constant variance
- No autocorrelation
- ACF shows no spikes

Bad residuals

- Drifting mean
- Growing spread
- ACF spikes remain
- Visible patterns



Good



Bad

How to Check Residuals

Three checks to run after fitting any model:

- **Plot residuals over time.** They should scatter randomly around zero with no visible trend, cycle, or growing spread.
- **ACF of the residuals.** All bars should be inside the noise zone. Any significant spike means the model missed structure at that lag — add more AR or MA terms.
- **Ljung-Box test.** A formal test for residual autocorrelation.
  $p > 0.05 \rightarrow$ no autocorrelation remaining, model is adequate.
  $p \leq 0.05 \rightarrow$ autocorrelation still present, model needs work.

All three checks should pass before you trust your model's forecasts.

Residuals — What to Do When They Fail

If your residuals show problems:

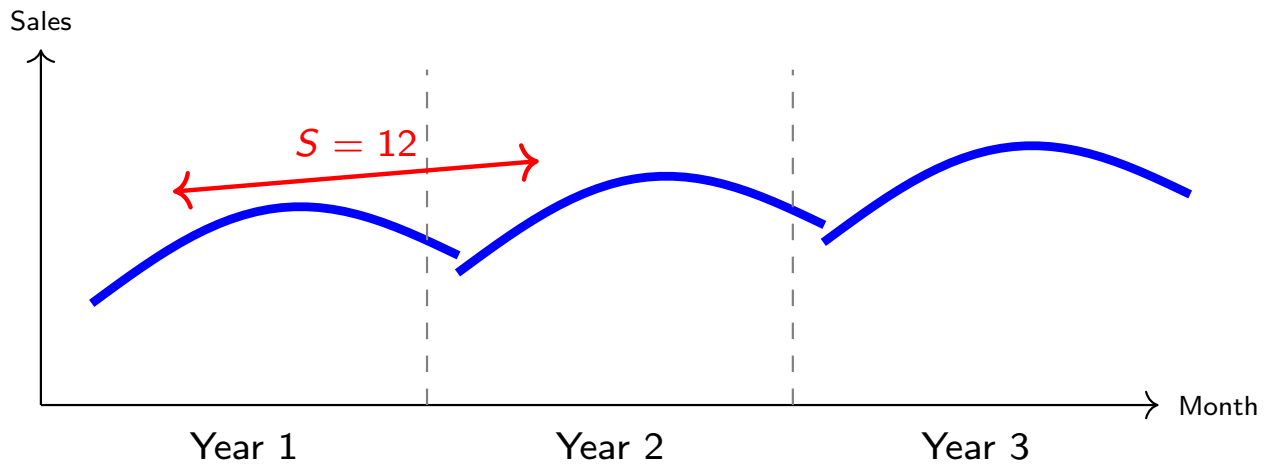
Problem	Fix
ACF spike at lag 1 or 2	Add MA or AR term
Slow decay in residual ACF	Difference the series again
Spike at lag S (e.g. 12)	Add seasonal MA or AR term
Variance grows over time	Apply log transform first
Mean not zero	Add a constant to the model

The cycle:
Fit model $\rightarrow$ check residuals $\rightarrow$ diagnose $\rightarrow$ adjust $\rightarrow$ repeat until residuals look like white noise.

Seasonality

A **regular pattern** that repeats every S time periods.

- Monthly data: $S = 12$ (pattern repeats every year)
- Quarterly data: $S = 4$
- Daily data with weekly pattern: $S = 7$



The same pattern (rise then fall) repeats every year, but the overall level may trend upward.

Seasonal Differencing

Regular differencing removes trend: $x'_t = x_t - x_{t-1}$

Seasonal differencing removes the seasonal pattern:

$$x'_t = x_t - x_{t-S}$$

Example: With monthly data ($S = 12$):

$$x'_t = x_t - x_{t-12}$$

"Compare this January to last January"

You may need both:

1. Seasonal differencing to remove seasonal pattern
2. Regular differencing to remove any remaining trend

The order usually matters less than doing both when needed.

SARIMA Notation

$$\text{ARIMA}(p, d, q) \times (P, D, Q)_S$$

	Non-seasonal		Seasonal
p	AR order	P	Seasonal AR order
d	Differencing	D	Seasonal differencing
q	MA order	Q	Seasonal MA order
		S	Seasonal period

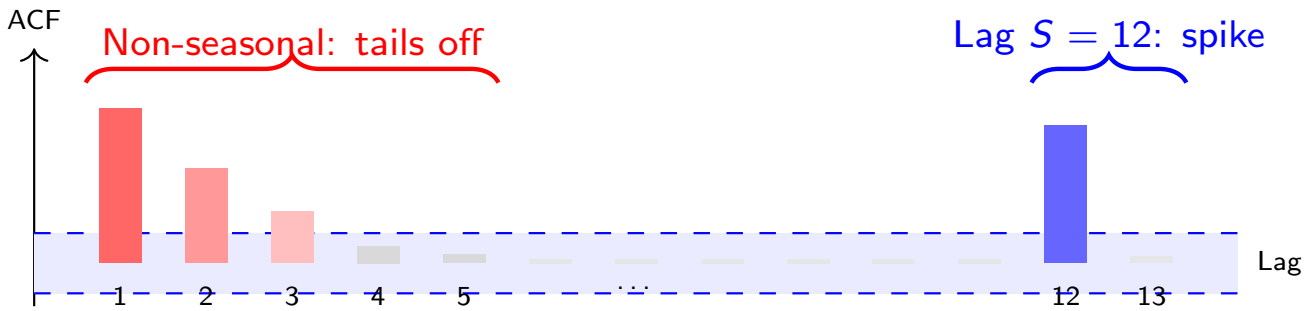
Example: $\text{ARIMA}(1, 0, 0) \times (0, 1, 1)_{12}$

- **Non-seasonal:** AR(1) — current value depends on previous value
- **Seasonal:** Seasonal difference ($D = 1$) and seasonal MA(1) at lag 12
- **Period:** $S = 12$ (monthly data)

Same ACF/PACF logic at **two scales**: early lags for non-seasonal, multiples of S for seasonal.

Reading ACF/PACF for Seasonal Models

Strategy: Examine two regions of the ACF/PACF separately.



How to read this:

Early lags tail off slowly $\rightarrow$ non-seasonal AR or ARMA (check PACF to confirm).

Lag 12 spikes sharply then cuts off $\rightarrow$ seasonal MA(1) component.

Candidate: ARIMA$(p, d, q) \times (0, D, 1)_{12}$ — use PACF to pin down p and q.

Why Random Splitting Fails for Time Series

Days 1–3: Cross-Sectional

- Rows are independent
- Order does not matter
- Random split is safe

Time Series

- Observations are ordered
- Past drives the future
- Random split is **wrong**

What goes wrong: Data Leakage

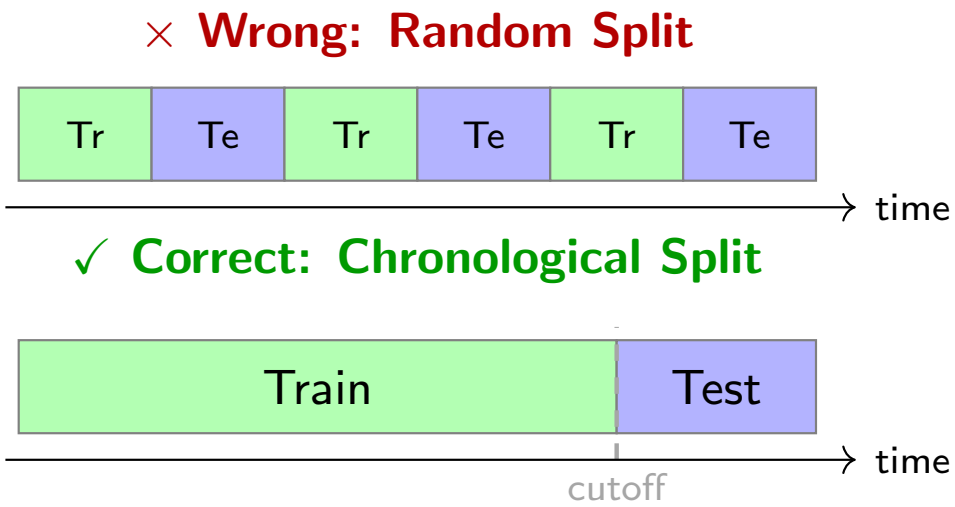
A random split may train on **month 18** and test on **month 3**.

The model has seen the future during training — results look good but collapse in real deployment.

Core rule: Always train on the **past**. Always test on the **future**. Never shuffle time.

The Correct Approach: Chronological Split

Always train on the past. Always test on the future.



Typical split sizes:

- A common choice is **80% train / 20% test**, but this depends on your data length and how far ahead you need to forecast
- The test set should be **at least as long** as your forecast horizon — if you want to forecast 12 months ahead, hold out at least 12 months
- Choose your cutoff date **before** you look at the results

The Limitation of a Single Split

A single train/test split works, but it has a weakness.

The problem

Your test set covers only **one fixed window** of time. If that window happens to be unusual — a recession, a pandemic, an unusually calm period — your error estimate is not representative of how the model performs in general.

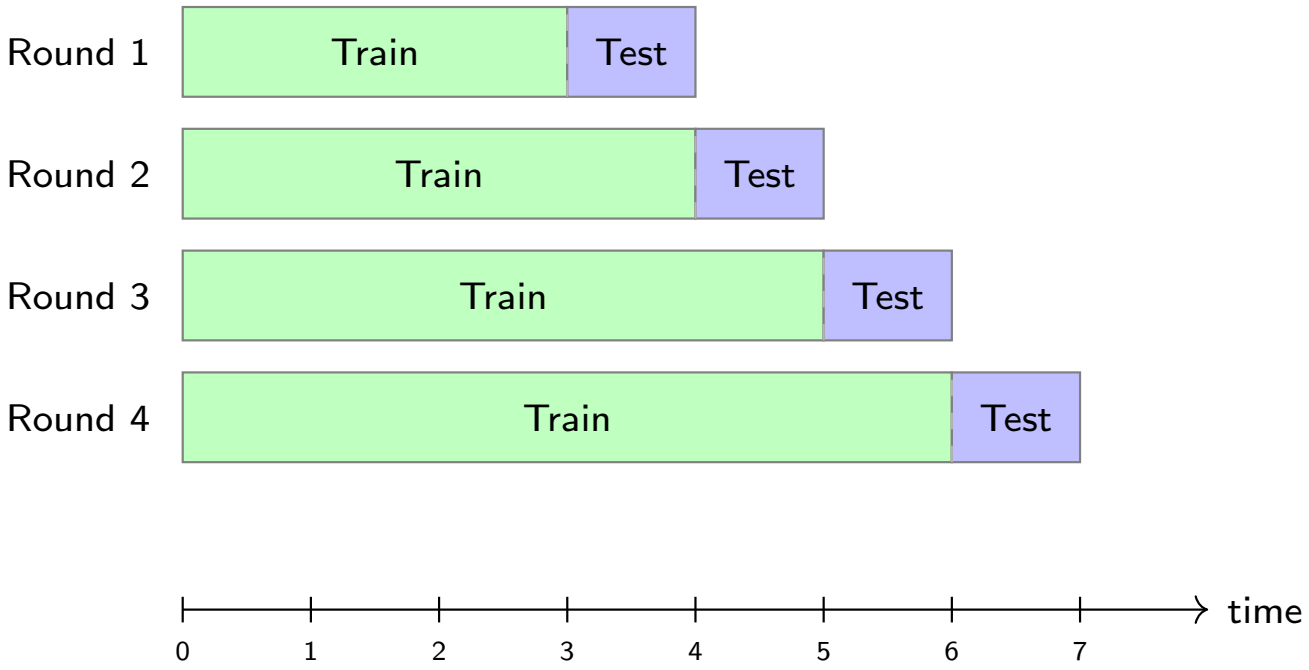
Example: You train on 2018–2022, test on 2023.

If 2023 was an extraordinary year, your test error will either look much better or much worse than what you should actually expect.

Solution: Evaluate the model across **multiple** time windows, not just one. This is called **walk-forward validation**.

Walk-Forward Validation

Simulate how the model performs in real time, round by round.



- Each round, the **training window grows** by one step
- The **test window** always looks one step ahead into unseen data
- The model is re-fitted from scratch at every round

Walk-Forward Validation vs. Cross-Validation

You have seen k-fold cross-validation on Days 1–3. This is the time series equivalent — same spirit, different rules.

	k-Fold CV	Walk-Forward
Split type	Random	Chronological only
Data shuffled?	Yes	Never
Training size	Fixed	Grows each round
Future leakage	Possible	Impossible by design
Use case	Cross-sectional	Time series

Key takeaway: Every principle you learned about model evaluation still applies — hold out data, measure error on unseen observations, avoid leakage. The only change is that for time series, **time cannot be ignored**. The past trains the model. The future tests it. Always.

Regression with Time Series: The Problem (1/2)

On Days 1–2: You learned regression $y = f(x)$ with independent observations.

But what if both y and x are time series?

The Problem

If you run ordinary regression on two time series:

1. The residuals will likely be **autocorrelated**
2. Your p-values and standard errors are **wrong**
3. You may find "significant" relationships that don't exist (**spurious regression**)

Regression with Time Series: The Problem (2/2)

Example: Regress ice cream sales on drowning deaths over time.

Both increase in summer $\rightarrow$ spurious correlation due to a shared seasonal pattern.

Why does this happen? Both series share a common trend or seasonal pattern. Ordinary regression mistakes this shared movement for a real relationship between the two variables.

Solution: Use **ARIMAX** or **SARIMAX** — models that combine regression with external variables *and* account for the time series structure in the errors.

ARIMAX and SARIMAX

ARIMAX — ARIMA with eXogenous variables

$$y_t = \underbrace{\beta_0 + \beta_1 x_t}_{\text{regression part}} + \underbrace{N_t}_{\text{ARIMA part}}$$

x_t = external predictor, N_t = ARIMA noise process

SARIMAX — adds Seasonal component

Same idea, but N_t is a **SARIMA** process — handles both the regression relationship and seasonal autocorrelation simultaneously.

Key insight: The regression tells you *how x affects y*. The ARIMA part tells you *how y correlates with its own past*. Both happen at the same time in one model.

ARIMAX: A Concrete Example

Example: Predicting monthly electricity demand.

Variable	**Role**
Electricity demand y_t	What we want to predict
Temperature x_t	External predictor (hotter = more AC use)
Past demand y_{t-1}, y_{t-2}	Captured by the ARIMA part

The model does two things simultaneously:

- **Regression part:** Every 1°C rise in temperature adds β_1 units of demand
- **ARIMA part:** Today's demand also depends on recent past demand and past prediction errors

ARIMAX: Why Not Plain Regression? (1/2)

Example: Predicting electricity demand from temperature.

What goes wrong

After fitting plain regression, look at the residuals:

- Today's error is similar to yesterday's — the residuals are **not random**
- This happens because yesterday's demand influences today's — a pattern the regression never accounted for
- When residuals are not random, p-values and confidence intervals **cannot be trusted**

ARIMAX: Why Not Plain Regression? (2/2)

What ARIMAX does instead:

1. Fit the regression between demand y_t and temperature x_t
2. Examine the residuals — fit an **ARIMA model on those residuals** to capture the remaining time series pattern
3. Re-estimate everything together so the regression coefficients are now correct

Key point: The ARIMA part does not replace your real variables — temperature is still in the model. The ARIMA part only handles the **leftover autocorrelation in the residuals** that plain regression missed.

CCF — The "Leading Indicator" Test (1/2)

What is the CCF?

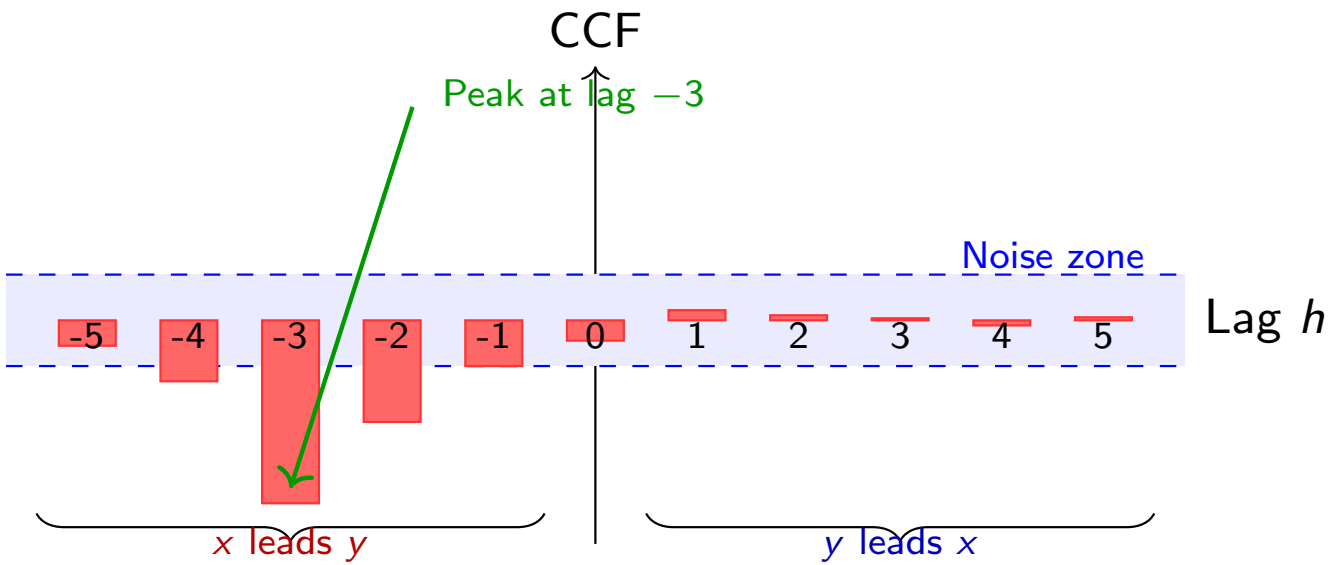
The CCF measures the correlation between **two different** series x and y at various lags.

Intuition: x = Rain, y = Umbrella Sales.

- Rain must fall *before* umbrella sales rise — there is a **delay**.
- The CCF asks: *"Does what happens to x today predict y later?"*
- If yes — x is a **leading indicator** of y.

Negative lag	x leads y	(x happens first)
Positive lag	y leads x	(y happens first)
Zero lag	both move simultaneously	

CCF — Reading the Plot (2/2)



Reading this plot: The strongest spike is at lag -3. This means x today predicts y three days later — x is a useful forecasting variable.

Lagged Predictors in ARIMAX (1/2)

Sometimes the effect of x on y is delayed.

$$y_t = \beta_0 + \beta_1 x_{t-d} + \beta_2 x_{t-d-1} + \cdots + N_t$$

Where d is the delay identified from the CCF, and N_t is an ARIMA process.

Example: Southern Oscillation Index (SOI) and fish recruitment.

- SOI measures Pacific ocean temperature anomalies
- CCF shows peak correlation at lags 5–10
- Ocean conditions today affect fish populations 5–10 months later
- Model: $y_t = \beta_0 + \beta_1 x_{t-5} + \cdots + \beta_6 x_{t-10} + N_t$

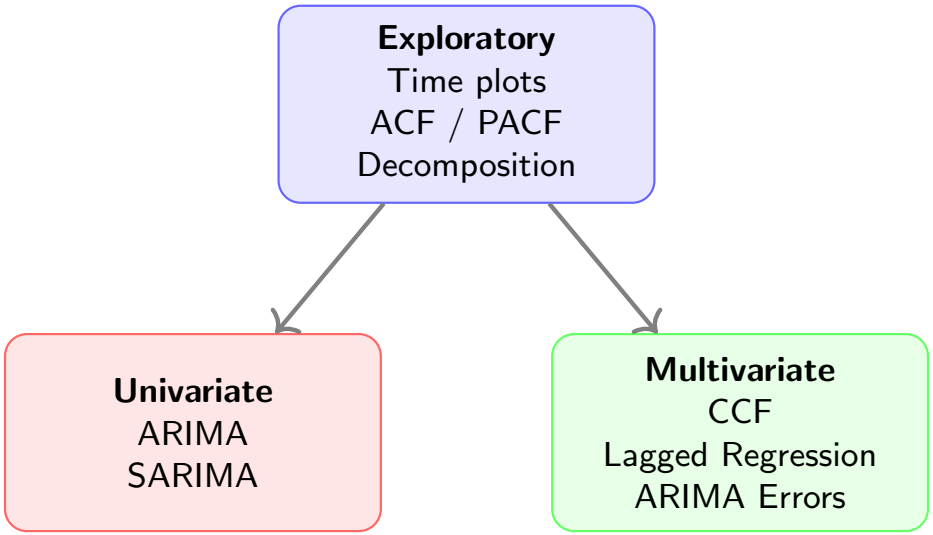
Lagged Predictors in ARIMAX (2/2)

Workflow for building a lagged ARIMAX model:

1. **Plot the CCF** between x and y to identify the delay d
2. **Build the lagged regression:** include $x_{t-d}, x_{t-d-1}, \ldots$ as predictors
3. **Check the residuals** — plot them and compute their ACF
4. **If the residual ACF shows spikes:** the regression alone missed some time series structure — fit an ARIMA model on those residuals and re-estimate everything together. This combined model is **ARIMAX**
5. **If data is also seasonal:** use SARIMA on the residuals instead — this gives you **SARIMAX**

Remember: The CCF tells you *when* x affects y. The ARIMA part captures *leftover autocorrelation in the residuals*. Your real dataset variables stay in the model throughout.

Your Time Series Toolkit



Workflow in practice:

1. **Explore:** Plot, decompose, check ACF/PACF
2. **Model:** Fit ARIMA (or SARIMA if seasonal)
3. **Relate:** If you have multiple series, use CCF + lagged regression
4. **Forecast:** Generate predictions with confidence intervals

Connecting to Machine Learning

You now know both classical ML (Days 1–3) and classical time series. In practice, they combine:

Feature Engineering

Use today's concepts as ML features:

- **Lag features:** $x_{t-1}, x_{t-2}, \ldots$
- **Rolling stats:** moving mean, std
- **Seasonal dummies:** month, weekday
- **Trend:** time index

Feed into Random Forest, XGBoost, etc.

Deep Learning

Neural networks for sequences:

- **RNNs:** Step-by-step processing
- **LSTMs:** Long-term dependencies
- **Transformers:** Attention-based

Learn AR/MA structure automatically.

ARIMA wins: Small data, interpretability, prediction intervals.
ML/DL wins: Large data, complex nonlinear patterns, many features.

Evaluating Forecasts: Why One Metric Is Not Enough

You've built a model and generated forecasts. How good are they?

Absolute metrics

MAE, MSE, RMSE

Measure error in the **same units** as the data.

"How many units off am I?"

Relative metrics

MAPE, SMAPE

Measure error as a **percentage** of the true value.

"What fraction did I miss?"

Running example used throughout:

$y_{\text{true}} = [10, 10, 10], \quad y_{\text{pred}} = [9, 10, 20]$

Errors: $[-1,\ 0,\ +10]$ — two small mistakes and one large one.

MAE — Mean Absolute Error

أكاديمية كاوست
KAUST ACADEMY

Formula

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

Take the absolute value of each error, then average them.

On our example:

$$\text{MAE} = \frac{|-1| + |0| + |10|}{3} = \frac{1+0+10}{3} = \mathbf{3.67}$$

Strengths:

- Same units as the data
- Easy to interpret
- Not dominated by large errors

Limitations:

- Treats all errors equally
- Does not highlight large mistakes

MAE answers: *"On average, how far off is my forecast?"*

MSE — Mean Squared Error

أكاديمية كاوست
KAUST ACADEMY

Formula

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

Square each error before averaging — large errors get penalised much more heavily.

On our example:

$$\text{MSE} = \frac{(-1)^2 + 0^2 + 10^2}{3} = \frac{1 + 0 + 100}{3} = \mathbf{33.67}$$

Strengths:

- Heavily penalises large errors
- Used as training loss in ML

Limitations:

- Units are **squared** — hard to interpret
- One large error dominates everything

RMSE — Root Mean Squared Error

Formula

$$\text{RMSE} = \sqrt{\text{MSE}} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

Take the square root of MSE to bring units back to the original scale.

On our example:

$$\text{RMSE} = \sqrt{33.67} = \mathbf{5.80}$$

Strengths:

- Same units as the data
- Still penalises large errors
- Most widely reported metric

Limitations:

- Still sensitive to outliers
- Less robust than MAE

MAE vs RMSE — What the Gap Tells You

MAE and RMSE always satisfy: $\text{MAE} \leq \text{RMSE}$

Situation	MAE	RMSE
Errors are all similar size	3.5	3.6
One large error among small ones	3.67	5.80
One very large outlier error	3.67	12.4

How to read the gap:

$\text{MAE} \approx \text{RMSE} \rightarrow$ errors are **consistent** — no single prediction is much worse than others.

$\text{RMSE} \gg \text{MAE} \rightarrow$ **outlier errors** exist — a few predictions are very far off.

MAPE — Mean Absolute Percentage Error (1/2)

أكاديمية كاوست
KAUST ACADEMY

Formula

$$\text{MAPE} = \frac{100\%}{n} \sum_{i=1}^{n} \frac{|y_i - \hat{y}_i|}{|y_i|}$$

Express each error as a percentage of the true value, then average.

Example: $y_{\text{true}} = [100,\ 100,\ 1]$, $y_{\text{pred}} = [90,\ 100,\ 2]$

Obs.	Error	True value	% Error
1	10	100	10%
2	0	100	0%
3	1	1	**100%**

$$\text{MAPE} = \frac{10\% + 0\% + 100\%}{3} = \mathbf{36.7\%}$$

MAPE — Mean Absolute Percentage Error (2/2)

Why the third observation caused 100% error:

The true value was 1, the prediction was 2 — an error of just 1 unit. But as a percentage of the true value, that is 100%.

Strengths:

- Scale-independent
- Easy to compare across datasets
- Popular in business forecasting

Limitations:

- **Explodes** when true value is near zero
- Undefined when true value is zero
- Biased toward under-prediction

Avoid MAPE when your data contains values close to zero — sensor readings, counts, or any process that can reach zero.

SMAPE — Symmetric Mean Absolute Percentage Error

Formula

$$\text{SMAPE} = \frac{100\%}{n} \sum_{i=1}^{n} \frac{2\,|y_i - \hat{y}_i|}{|y_i| + |\hat{y}_i|}$$

Uses the average of the true and predicted value in the denominator — more stable near zero.

Same example: $y_{\text{true}} = [100,\ 100,\ 1], \quad y_{\text{pred}} = [90,\ 100,\ 2]$

$$\text{SMAPE} = \frac{10.5\% + 0\% + 66.7\%}{3} = \mathbf{25.7\%}$$

Better than MAPE near zero? Yes — observation 3 gives 66.7% instead of 100%. Still large, but no longer explosive.

Trap: A large absolute error (e.g. 100 units off) can produce a surprisingly small SMAPE. Always pair with MAE or RMSE.

Choosing the Right Metric (1/2)

Metric	Best For	Watch Out
MAE	General default	Treats all errors equally
MSE	Training loss	Squared units, outlier-sensitive
RMSE	Reporting	Still outlier-sensitive
MAPE	Business forecasting	Explodes near zero
SMAPE	Near-zero values	Large errors can look small

Choosing the Right Metric (2/2)

Practical rules:

- Always report **at least two metrics** — one absolute (MAE or RMSE) and one relative (MAPE or SMAPE)
- If MAE $\approx$ RMSE: your errors are consistent across predictions
- If RMSE $\gg$ MAE: a few predictions are very far off — investigate those cases
- If your data has values near zero: skip MAPE entirely, use SMAPE + MAE

No single metric tells the full story. Each one highlights a different aspect of your model's performance — use them together.

References

- Penn State STAT 510: Applied Time Series Analysis
  https://online.stat.psu.edu/stat510/
- Shumway, R.H. & Stoffer, D.S., *Time Series Analysis and Its Applications With R Examples*, 5th Edition, Springer, 2025.
- Hyndman, R.J. & Athanasopoulos, G., *Forecasting: Principles and Practice*, 3rd Edition
  https://otexts.com/fpp3/

Slides contributed by Yazan Alshoibi